

Prerequisites

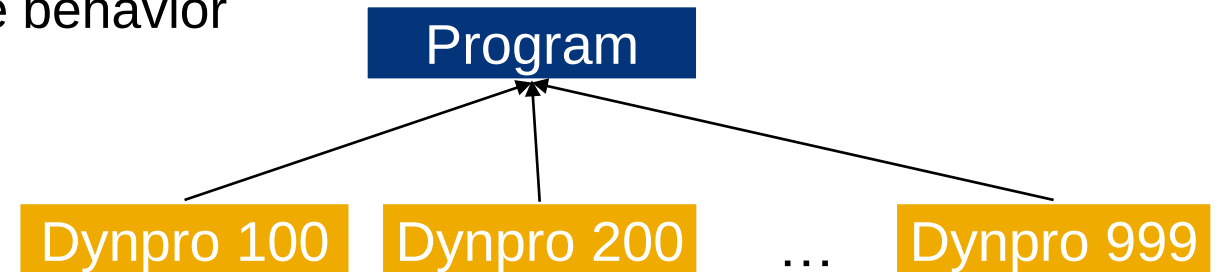
- Fundamental programming skills
- Successful completion of the chapter *Introduction to SAP Application Development*
- Recommended: Completion of the chapters *Development of a first application*, *Basic concepts of ABAP programming*, and *Database access with ABAP programs*

Agenda

- I. Fundamentals**
- II. Dynpro at runtime**
- III. Dynpro layout**
- IV. Developing Dynpros**
- V. Deploying and accessing Dynpros**

General information about Dynpros

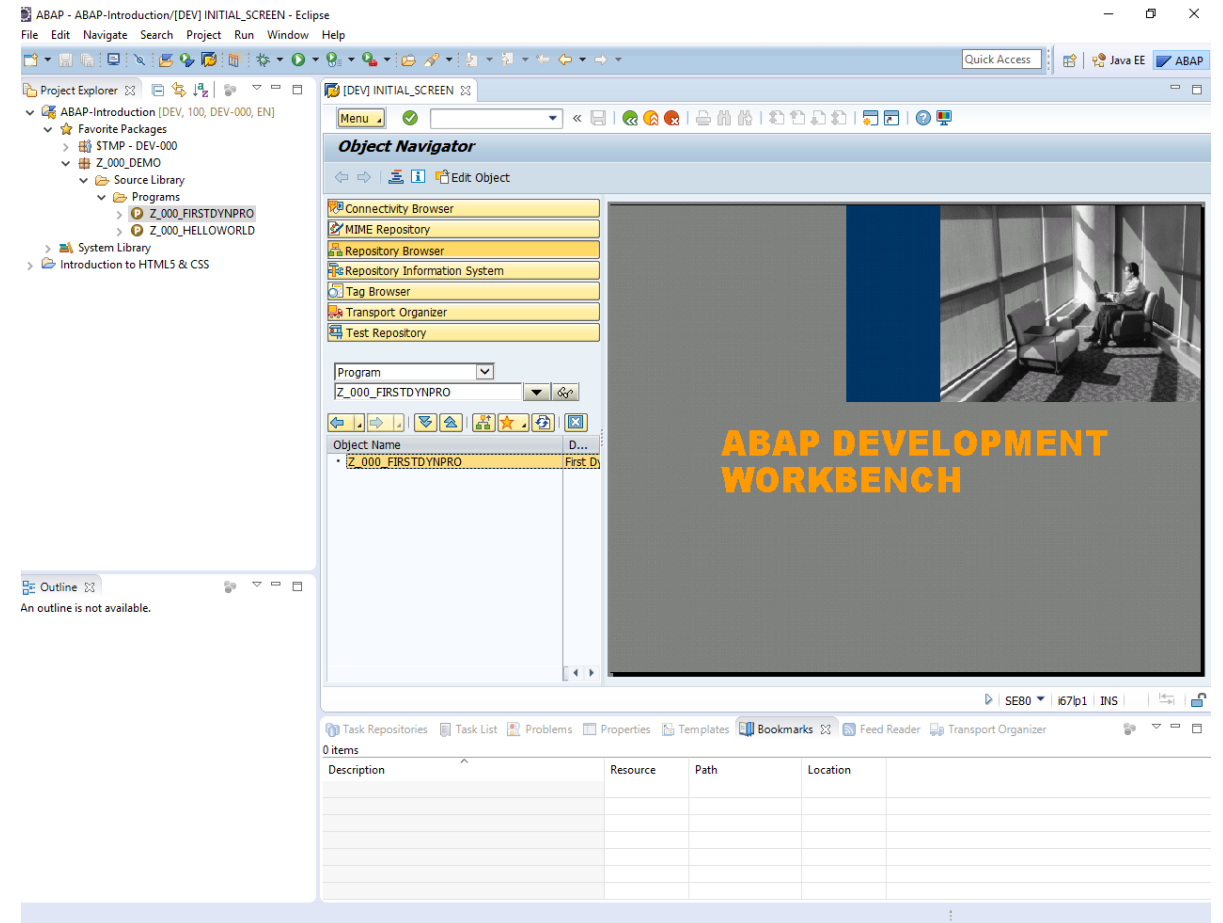
- Dynpro = Dynamic program / Dialog program
- Dynpros contain program logic combined with a Graphical User Interface (GUI)
- A program can contain several Dynpro Screens
- Every Dynpro has a unique ID in a program
- Dynpros use their own instructions for coordinating runtime behavior → **this is no ABAP**
- But: **Dynpros call modules which are implemented in ABAP**
- Modules called by a Dynpro control the runtime behavior



I. Fundamentals

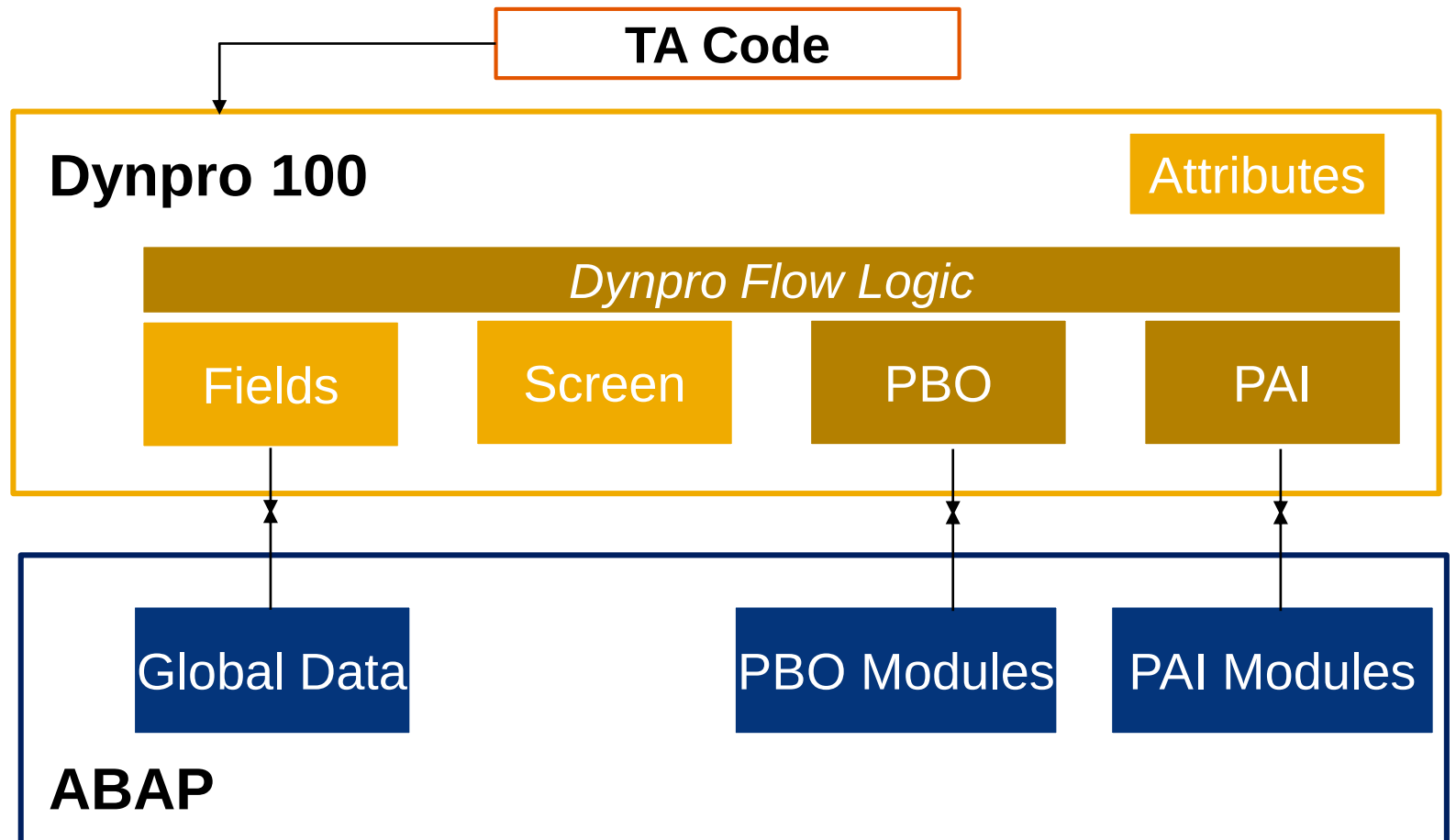
Dynpro development in Eclipse

- Dynpros are specially designed for a usage within the traditional SAP GUI
- The integrated SAP GUI in Eclipse can be used to create Dynpros
- Dynpros call ABAP modules which can be implemented in Eclipse
- Advantage: Coding in Eclipse is more comfortable compared with the traditional SAP GUI
- All functions which are not available in Eclipse can be used via the integrated SAP GUI



Dynpro Structure

- **Transaction Code:** Start Dynpro
- **Fields:** data objects of a Dynpro
- **Screen:** UI defined with the screepainter^(*)
- **Attributes:** Settings for the screen
- **PBO:** Process before output (before screen is displayed)
- **PAI:** Process after Input (after the user input)



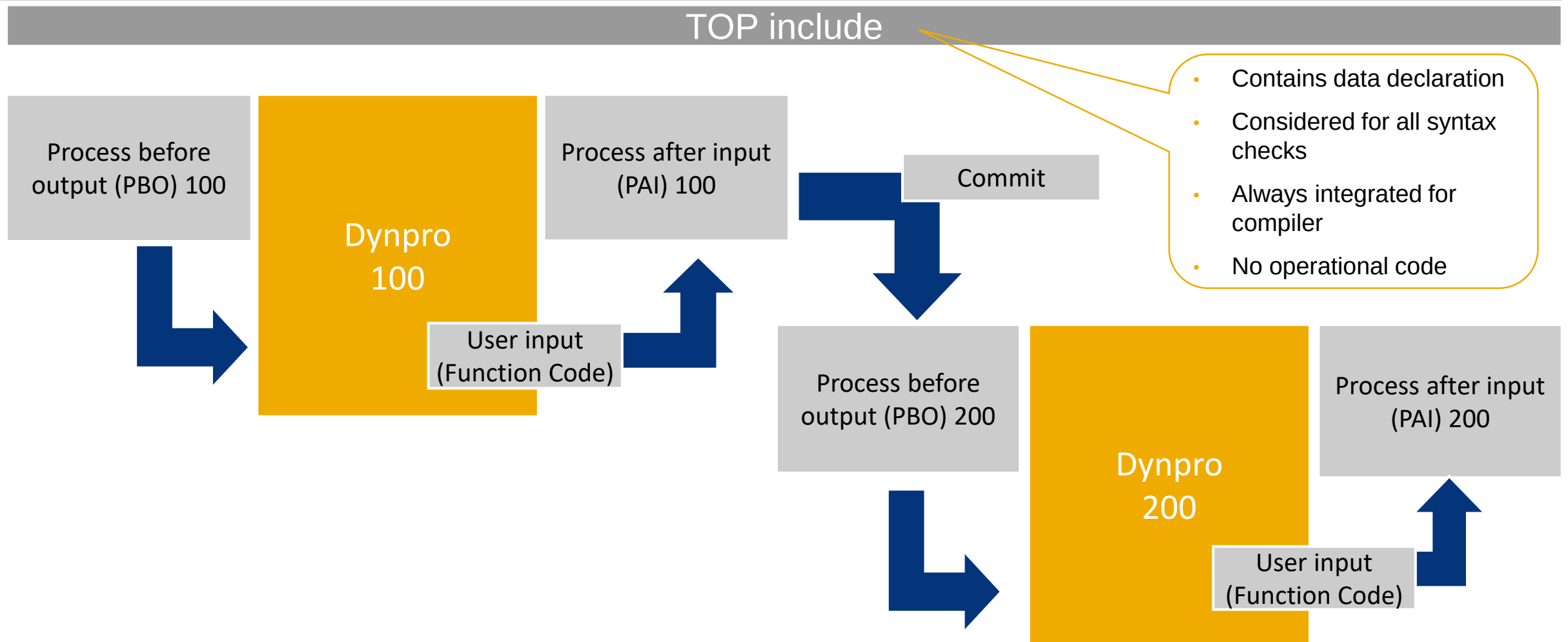
(*) The screenpainter is only available within sap gui for windows. With MAC you have to use the alphanumeric layout editor.

Usage see https://help.sap.com/doc/saphelp_nw73ehp1/7.31.19/en-US/d1/80231e454211d189710000e8322d00/content.htm?no_cache=true

II. Dynpro at runtime

Dynpro runtime modules

Dynpro Flow Logic



Coordinates Runtime behavior

- PROCESS BEFORE OUTPUT
 - Executed **before sending the Dynpro** screen
 - Used to prepare the screen, e.g. set default values
- PROCESS AFTER INPUT
 - Executed every time a user interaction triggers a **function code / OK code**
 - Example: 
- PROCESS ON HELP-REQUEST
 - Optional
 - Triggered when user selects the **info help F1**
- PROCESS ON VALUE-REQUEST
 - Optional
 - Triggered when user selects the **input help F4**

Code Example:

```
PROCESS BEFORE OUTPUT.  
MODULE STATUS_0200.
```

```
PROCESS AFTER INPUT.  
MODULE USER_COMMAND_0200.
```


- **MODULE:**
 - Call an ABAP Module
- **FIELD:**
 - Dynpro data object
- **CHAIN / ENDCHAIN**
 - Dynpro data object
 - Related to global data object of the ABAP program
- **LOOP / ENDLOOP:**
 - Loop over Dynpro controls (e.g. buttons, input fields, ...) -> e.g. change properties in PBO
- **CALL SUBSCREEN:**
 - Include a subscreen Dynpro

- Dynpros contain user interfaces which may contain the following elements:

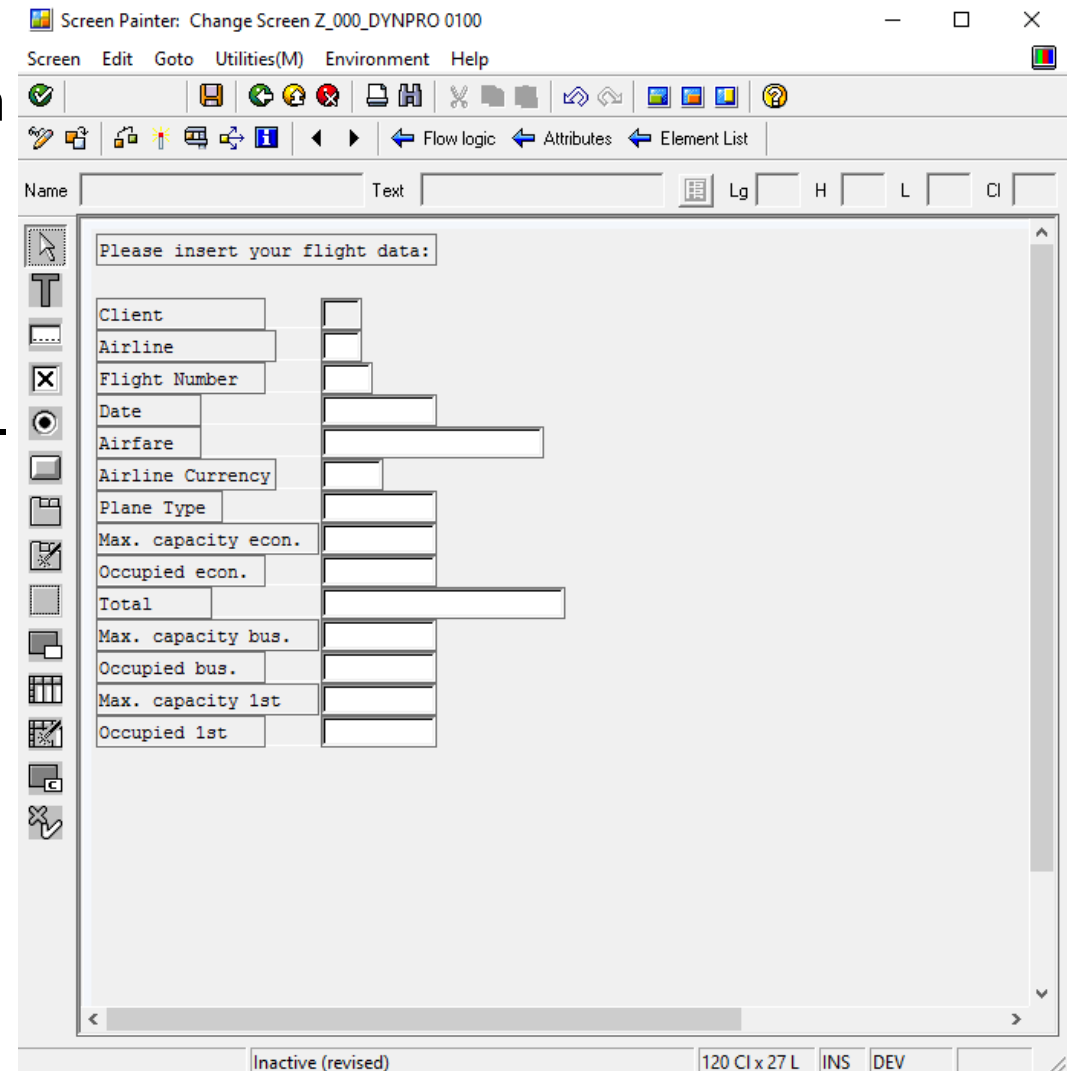
- Text
- Input fields
- Check boxes
- Radio buttons
- Buttons
- Tabs
- Frame Subscreens
- Table Controls
- Custom Controls



- Element's attributes may be changed during runtime (e.g. button text)

Graphical Screen Painter

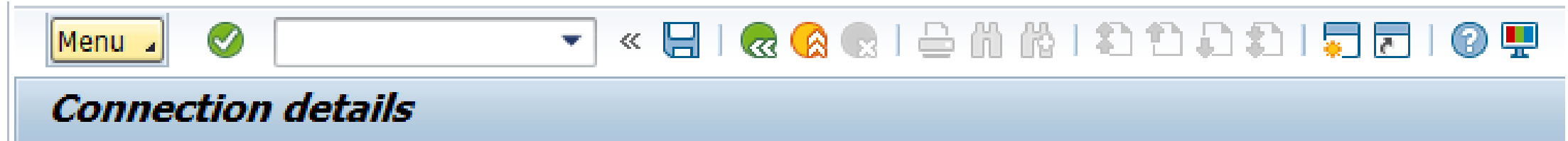
- Graphical Screen Painter is an external program which is integrated into the SAP GUI
- Can be used to draw a new Dynpro
- Elements can be referenced to the data dictionary (e.g. tables)
- Several views are available:
 - **Main view:** contains the main elements of your Dynpro
 - **Element list:** contains all element's attributes
 - **Dictionary:** contains elements from the data dictionary which may be inserted into your Dynpro
 - **Attributes:** contains attributes of one element from your Dynpro



- For each Dynpro you may define a title and status bar
- Title defines the title of the Dynpro (Example: *Connection details*)
- Status bar defines the menu and the flexible and fixed tool bar (Examples: *Save, Back, Exit, ...*)
- Status bar defines function code / return status (OK_CODE) of elements
- Based on the OK_CODE you define further actions (e.g save data, leave application, back to last screen ...)

Status

Title



Transaction codes

TA Code

- Transaction codes serve to directly access programs
- They can be used like the standard SAP transactions
- SAP users can be given or denied rights to the transactions
- They are defined based on a Dynpro program and a starting screen
- The starting screen of a program is called when the transaction code is executed (e.g. SE80 calls the starting screen of the ABAP Workbench)

Transaction code	Z_000_DYNPRO
Package	

Transaction text	First Dynpro
Program	Z_000_DYNPRO
Screen number	100
Authorization Object	 Values

☒ Maintenance of standard transaction variant allowed

Classification

Transaction classification

☒ Professional User Transaction

☐ Easy Web Transaction Service

☐ Pervasive enabled

GUI support

☒ SAPGUI for HTML

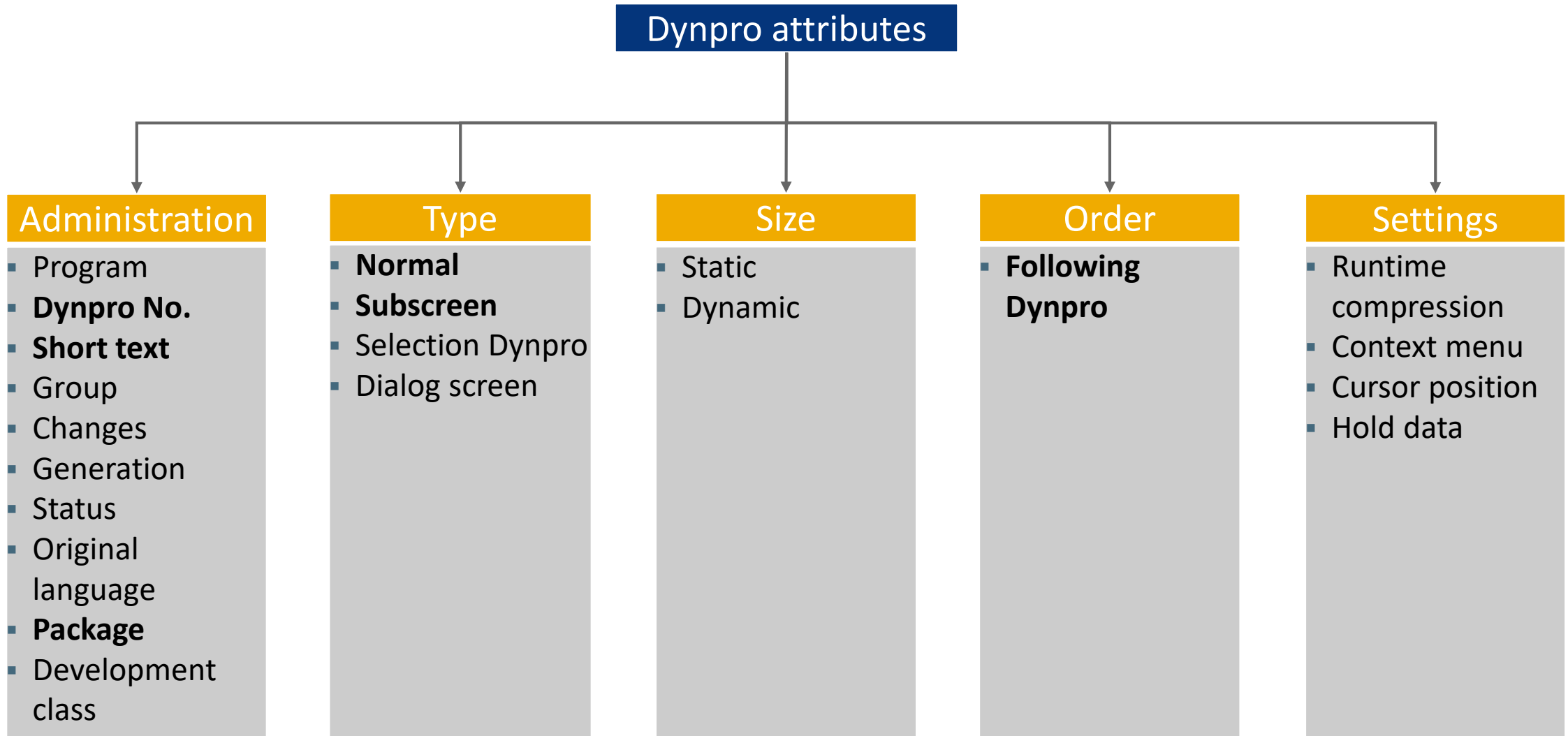
☐ SAPGUI for Java

☒ SAPGUI for Windows

III. Dynpro layout

Dynpro attributes

Attributes

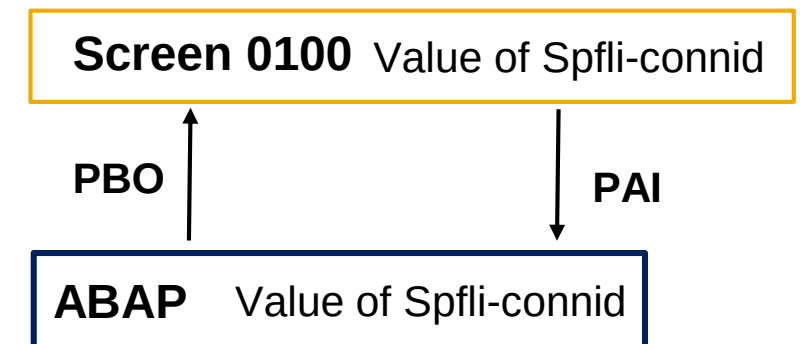
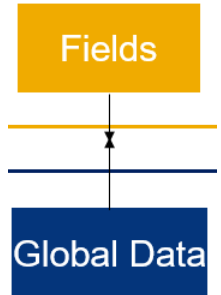
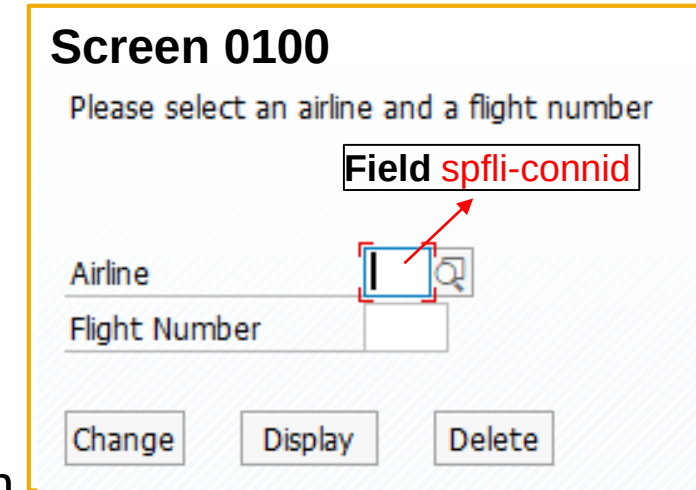


IV. Data management

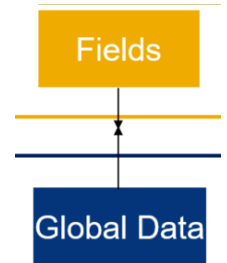
Dynpro Fields

Fields

- Fields are the **data objects of a dynpro**, e.g. input field
- In Dynpro **no explicit data declaration is possible** (*“you can not create a local variable”*)
- A Field is either related to the ABAP Dictionary or a global data object in the ABAP program
 - A Field is a **data handler** to pass the information from the ABAP layer into the Dynpro and back
- After the **PBO** phase the data from the global ABAP data object is sent to the related field
- After **PAI** the data from the field is sent back to the global data object

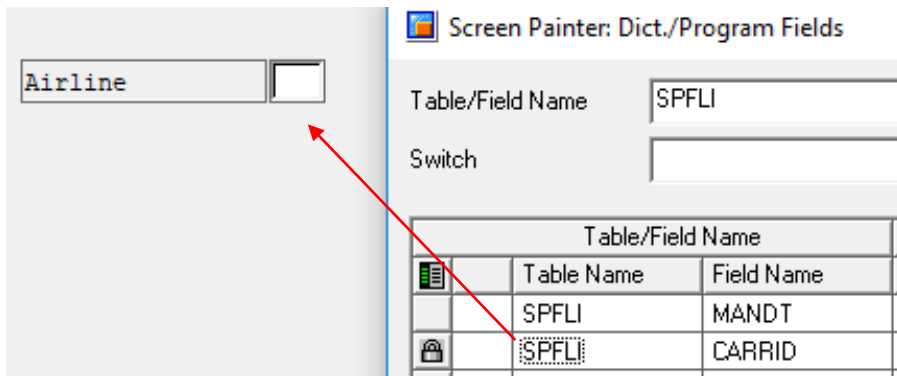


- If a Dynpro **Field** refers to a **structure of the ABAP dictionary** (see SPFLI in next exercise) ...
 - You have to explicitly define the structure via the **Tables statement** in the Dynpro
 - Otherwise the Dynpro Field and ABAP program are not connected => No data handling



Example:

Screen field refers to database table SPFLI



You have to declare SPFLI in the Top Include of your Dynpro application

```
*&-----  
*& Include Z_000_DYNPRO_TOP  
*&-----  
PROGRAM Z_000_DYNPRO_APPLICATION.  
  
TABLES spfli.
```

- Special Include file that contains all global declarations
- No operational code allowed
- In contrast to a regular include file, the top include is always considered by the syntax check and compiler
- In the context of Dynpro applications, the top include encapsulates all global variables, e.g.
 - **TABLES**
 - **OK_CODE**
- Accessible by every Dynpro in the program

```
*&-----  
*& Include Z_000_DYNPRO_TOP  
*&-----  
PROGRAM Z_000_DYNPRO_APPLICATION.  
  
TABLES spfli.  
  
DATA ok_code LIKE sy-ucomm.
```

- Special dynpro field that is implicitly defined when creating a new Dynpro
- Manages the return code of the Dynpro

1. Set the return code, based on the buttons in the status bar



2. Handle ok_code in PAI section

```
Case ok_code.  
  WHEN 'LEAVE'.  
    Leave Program.  
  WHEN 'BACK'.  
  WHEN 'SAVE'.  
    IF spfli <> wa_flight.  
      MODIFY spfli FROM spfli.  
    ENDCASE.
```

Now you know the fundamentals of a Dynamic Program (Dynpro).

To consolidate your knowledge, you can do tasks 1 and 2 of the Dynpro exercise.

IV. Developing Dynpros

Step 1: Determine Dynpro attributes

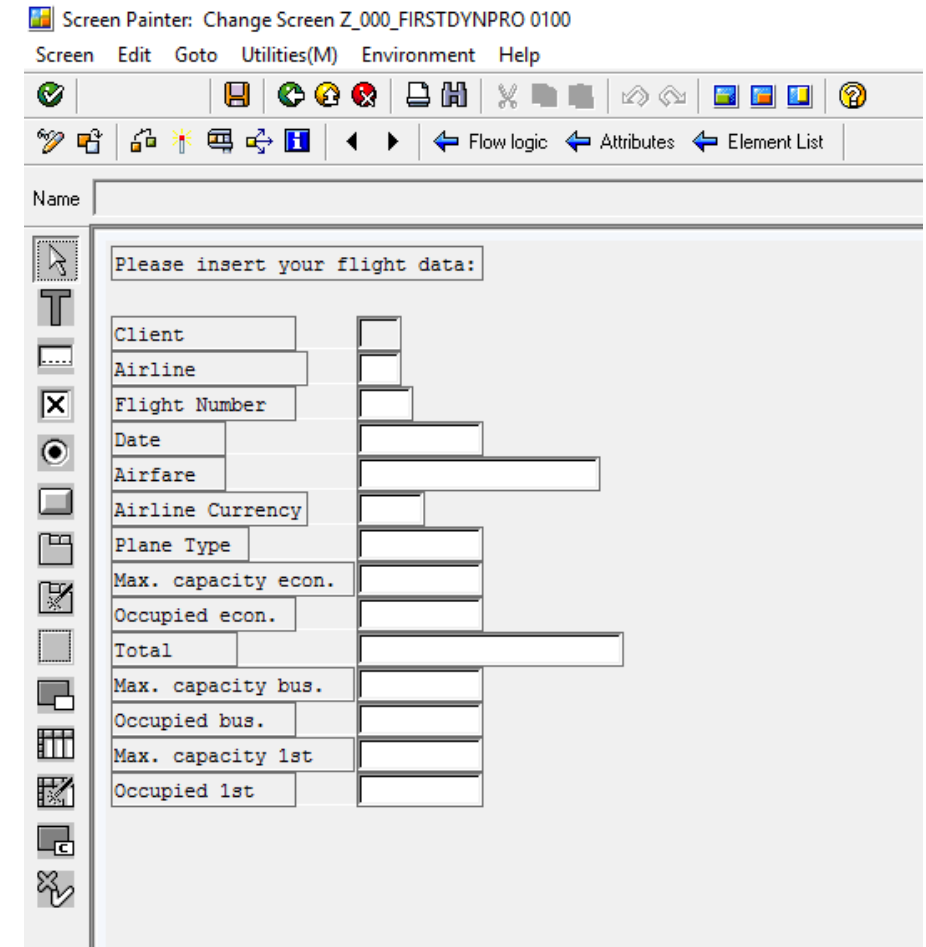
- The following Dynpro attributes have to be determined:
 - Screen number (3 digits)
 - Short Description
 - Dynpro Type (Normal, Subscreen, ...)
 - Next Screen
- The element list contains all attributes which are contained in the Dynpro
- The flow logic contains the program code for the PBO and PAI modules which control the Dynpro at runtime

The screenshot shows the 'Attributes' tab of the SAP Dynpro configuration interface. At the top, 'Screen number' is set to '100' and the title is 'New(Revised)'. The 'Attributes' tab is selected, with 'Element list' and 'Flow logic' also visible. The 'Short Description' field contains 'My first Dynpro'. Below it, 'Original Language' is set to 'EN' (English), and 'Package' is empty. 'Last Changed' and 'Last Generation' are both set to '00:00:00'. The 'Screen Type' section has four radio buttons: 'Normal' (selected), 'Subscreen', 'Modal dialog box', and 'Selection screen'. The 'Settings' section on the right contains five checkboxes: 'Hold Data', 'Switch Off Runtime Compress', 'Template - non-executable', 'Hold Scroll Position', and 'Without Application Toolbar', all of which are currently unchecked. The 'Other Attributes' section at the bottom includes 'Next Screen' set to '200', 'Cursor Position' (empty), 'Screen Group' (empty), 'Lines/Columns' with 'Occupied' set to '0' and 'Mainten.' set to '27', and 'Context Menu' set to 'FORM ON CTMENU'. A 'Properties' button is located at the bottom right.

IV. Developing Dynpros

Step 2: Create Dynpro layout

- Use the Graphical Screen Painter to create the elements within your Dynpro
- Maintain attributes for your Dynpro elements (data type, field size,...)
- Assign unique names to your GUI elements
- Connect input fields and tables to dictionary objects if necessary
- Assign a function code to buttons and other elements which create events
- Assign the OK_CODE to the Dynpro



IV. Developing Dynpros

Step 3: Code the processing logic

Dynpro Flow Logic

The flow logic defines PBO and PAI modules

PBO modules are called before the Dynpro is shown to the user

```
PROCESS BEFORE OUTPUT.  
MODULE STATUS_0200.  
  
PROCESS AFTER INPUT.  
MODULE USER_COMMAND_0200.
```

PAI modules are called after the user confirmed an input

ABAP Modules

```
MODULE STATUS_0200 OUTPUT.  
  SET PF-STATUS '200'.  
  SET TITLEBAR '200'.  
  
  spfli = wa_flight.  
  
  * Further assignments  
  * Preparation of menu functionalities  
  * ...  
  
ENDMODULE.  
  
MODULE USER_COMMAND_0200 INPUT.  
  
  Case ok_code.  
    WHEN 'LEAVE'.  
      Leave Program.  
    WHEN 'BACK'.  
    WHEN 'SAVE'.  
      IF spfli <> wa_flight.  
        MODIFY spfli FROM spfli.  
      ENDIF.  
    ENDIF.  
  
    * Further handling of user actions  
    * ...  
  
    CLEAR ok_code.  
  
  ENDCASE.  
  
ENDMODULE.
```

A PBO module usually defines title and status bar

You can also assign database values to Dynpro elements

Special user events can be handled

Database operations can be triggered

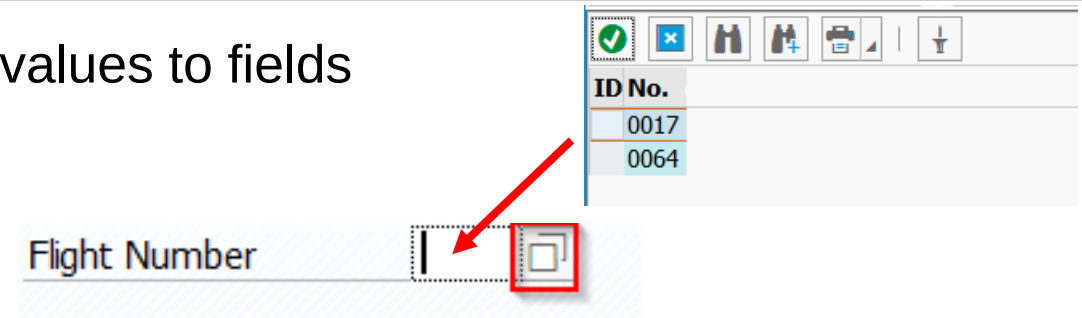
PAI modules usually handle the ok_code

Now you know the complete development process of a Dynamic Program (Dynpro).
To consolidate your knowledge, you can do task 1, 2, 3 and 4 of the Dynpro exercise.

V. Dynpro Usability

Input Help

- Input helps (F4) support the user when inserting values to fields
 - Avoid misspelling
 - Avoid invalid input



- Trigger the input help in the “**PROCESS ON VALUE-REQUEST**” section
- Implement module to return a value table

```
PROCESS ON VALUE-REQUEST.  
FIELD spfli-connid MODULE INPUT_HELP_0100.
```

- Self programmed input help not always necessary
 - Data dictionary based fields mostly provide usually have a default input help

Now you know the complete development process of a Dynamic Program (Dynpro).

To consolidate your knowledge, you can do task 5 of the Dynpro exercise.

Check your knowledge

Check your knowledge

- Why is it preferable to use Eclipse for developing Dynpros instead of the traditional SAP GUI?
- Which steps at developing Dynpros can be done in Eclipse without the integrated SAP GUI?
 - ☐ Determine Dynpro attributes
 - ☐ Create Dynpro layout
 - ☐ Coding PBO and PAI modules
- Which functionalities are usually contained within the PBO and PAI modules?
- Where do you define by default which Dynpro is called after the user input was confirmed?
 - ☐ Transaction code
 - ☐ Dynpro attributes
 - ☐ PAI module

Solution

Solution

- Why is it preferable to use Eclipse for developing Dynpros instead of the traditional SAP GUI?
See section *Fundamentals*
- Which steps at developing Dynpros can be done in Eclipse without the integrated SAP GUI?
 - ☐ Determine Dynpro attributes
 - ☐ Create Dynpro layout
 - ☒ Coding PBO and PAI modules
- Which functionalities are usually contained within the PBO and PAI modules?
See section *Developing Dynpros*
- Where do you define by default which Dynpro is called after the user input was confirmed?
 - ☐ Transaction code
 - ☒ Dynpro attributes (see chapter *Developing Dynpros*)
 - ☐ PAI module

© 2018 SAP SE or an SAP affiliate company. All rights reserved.

No part of this publication may be reproduced or transmitted in any form or for any purpose without the express permission of SAP SE or an SAP affiliate company.

SAP and other SAP products and services mentioned herein as well as their respective logos are trademarks or registered trademarks of SAP SE (or an SAP affiliate company) in Germany and other countries. Please see <http://global12.sap.com/corporate-en/legal/copyright/index.epx> for additional trademark information and notices.

Some software products marketed by SAP SE and its distributors contain proprietary software components of other software vendors.

National product specifications may vary.

These materials are provided by SAP SE or an SAP affiliate company for informational purposes only, without representation or warranty of any kind, and SAP SE or its affiliated companies shall not be liable for errors or omissions with respect to the materials. The only warranties for SAP SE or SAP affiliate company products and services are those that are set forth in the express warranty statements accompanying such products and services, if any. Nothing herein should be construed as constituting an additional warranty.

In particular, SAP SE or its affiliated companies have no obligation to pursue any course of business outlined in this document or any related presentation, or to develop or release any functionality mentioned therein. This document, or any related presentation, and SAP SE's or its affiliated companies' strategy and possible future developments, products, and/or platform directions and functionality are all subject to change and may be changed by SAP SE or its affiliated companies at any time for any reason without notice. The information in this document is not a commitment, promise, or legal obligation to deliver any material, code, or functionality. All forward-looking statements are subject to various risks and uncertainties that could cause actual results to differ materially from expectations. Readers are cautioned not to place undue reliance on these forward-looking statements, which speak only as of their dates, and they should not be relied upon in making purchasing decisions.